

# From approved tokens to a PayPal payout.

A revised proposal for Kwasi: a Devnet treasury receipt opens a controlled payout from ChainPay's prefunded PayPal sandbox account.

# PayPal now moves funds. ChainPay owns the gap.

The October 3 deck covered invoice bookkeeping. This revision adds a separate payout and a compensation workflow.

## Keep the existing payment rail

- Buyer-approved mandate limits token spending.
- The relay transfers Devnet tokens and creates the existing receipt PDA.
- Axum verifies the receipt and controls provider access.

## Add a treasury-backed payout

- ChainPay receives the test tokens.
- PayPal pays the merchant from a separate sandbox balance.
- Clear payout failure can trigger a controlled token return.

**New trust boundary:** the token leg is mandate-enforced; the PayPal leg and refund are enforced by ChainPay's backend and treasury controls.

# Test tokens do not become PayPal dollars.

This is a prefunded sandbox demonstration, with two independent balances and two distinct proofs.

## 1 · Solana Devnet

### Buyer → ChainPay treasury

Exact approved mint and base units. Finalized receipt proves the treasury received test tokens.

## 2 · PayPal sandbox

### ChainPay balance → Merchant

Exact signed fiat amount and currency. A matching payout item proves sandbox funds reached the recipient.

No token redemption, exchange or automatic peg is provided. Display the two amounts separately and sign the agreed demo terms.

# The receipt opens the payout gate.

Human approvals bind the draft, the terms, the buyer debit and the final bookkeeping entry.

|    |                                   |                                                                         |
|----|-----------------------------------|-------------------------------------------------------------------------|
| 01 | <b>Merchant approves draft</b>    | Verify the invoice-owning merchant account and its enrolled wallet.     |
| 02 | <b>Create + send invoice</b>      | Use merchant-authorized sandbox credentials; retain ID and revision.    |
| 03 | <b>Merchant signs terms</b>       | Bind treasury, payout recipient, exact amounts, fees and expiry.        |
| 04 | <b>Buyer approves + preflight</b> | Reserve the invoice intent; check terms, recipient and prefunding.      |
| 05 | <b>Transfer tokens</b>            | Existing mandate transfers to treasury and creates the receipt PDA.     |
| 06 | <b>Verify + claim receipt</b>     | Finalize evidence; attach receipt claim to the reserved invoice intent. |
| 07 | <b>Dispatch one payout</b>        | Axum rechecks invoice; sends from prefunded PayPal sandbox balance.     |
| 08 | <b>Merchant compares + signs</b>  | Invoice + treasury receipt + successful payout item, shown separately.  |
| 09 | <b>Record OTHER + read back</b>   | Verify one exact external-payment entry against the invoice.            |

Design sequence. Failure and unknown outcomes branch before bookkeeping; see slides 9–11.

# Bind the payout destination before taking tokens.

An invoice ID alone cannot prove that the treasury, recipient and exchange terms were approved.

| BINDING          | EXACT SIGNED FIELDS                                                                                  |
|------------------|------------------------------------------------------------------------------------------------------|
| Identity + scope | Sandbox environment, invoice owner, invoice ID + revision, merchant wallet, buyer owner, expiry.     |
| Token leg        | Devnet cluster, program, mint, token program, decimals, integer base units and treasury destination. |
| PayPal leg       | Recipient ID/type, exact fiat amount + currency, fee allocation and agreed demo exchange terms.      |
| Recovery terms   | Same-buyer, same-token, exact-amount compensation; pending/unknown means wait.                       |

Use a versioned digest of the **complete binding** as the request hash. Reuse the existing instruction only after verifying compatibility.

Design requirement derived from the proposed two-leg flow; no new Anchor instruction is a compatibility target, not a verified conclusion.

# Check first. Keep the failure path anyway.

Preflight reduces avoidable failures; it cannot reserve PayPal funds or freeze an invoice paid elsewhere.

## Before the token debit

- Invoice still open, unchanged and fully unpaid.
- Known sandbox merchant and enrolled recipient.
- Signed terms current; reserve the invoice before debit.
- Prefunding headroom covers payouts and fees.

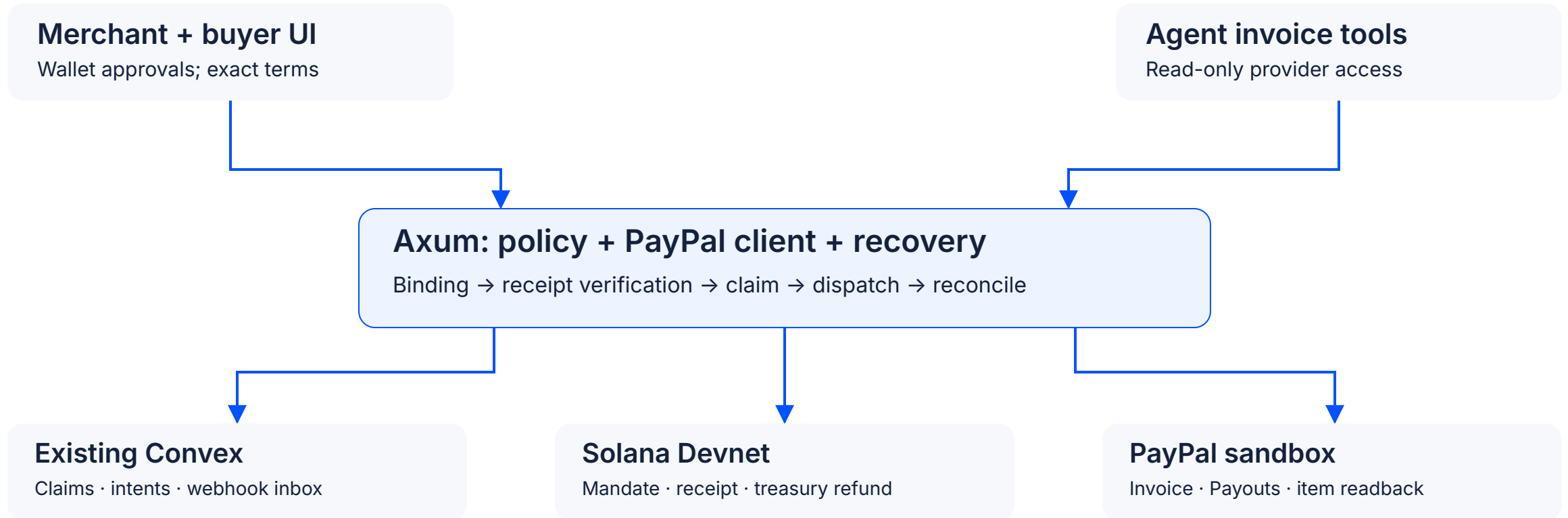
## What this cannot promise

- Balance API data can lag up to three hours.
- A recipient can still become blocked or held.
- Hosted invoice checkout can race the payout.
- Two systems do not share an atomic transaction.

Demo scope: controlled accounts and one invoice at a time. Still detect outside payments and route conflicts to explicit review.

# One Axum policy layer controls both external effects.

Agents can inspect invoices and receipts. They cannot call PayPal Payouts or the treasury refund signer.



Existing Axum + Convex follows the approved October 2 scope amendment. Verify deployment identity and data continuity; add no second backend.

# Provider idempotency is a window, not the ledger.

A receipt-derived key helps only when the whole operation is durably claimed and its intent cannot change.

---

## Persist before I/O

Store the signed intent, canonical payload, invoice + receipt claims, sender IDs and dispatch state before the PayPal POST.

---

## 30-day window

PayPal deduplicates `sender_batch_id` within 30 days. A duplicate response points to the original batch. Keep local uniqueness beyond this window.

---

## Unknown response

Reconcile the original operation. Any supported retry uses the same payload and keys within retention; never create a new key to “try again.”

# An accepted batch does not mean the merchant was paid.

Only the matching item's current state determines the payout outcome.

| OBSERVED PAYOUT STATE        | MEANING                       | CHAINPAY ACTION                                     |
|------------------------------|-------------------------------|-----------------------------------------------------|
| Item SUCCESS                 | Recipient credited            | Enable signed invoice reconciliation.               |
| Item FAILED                  | No sender deduction           | Persist failure proof; evaluate token compensation. |
| Item RETURNED                | Funds returned                | Verify correlation and return before compensation.  |
| PENDING / UNCLAIMED / ONHOLD | Not resolved                  | Poll or await verified event. No refund.            |
| BLOCKED / unknown response   | Insufficient no-payment proof | Reconcile or review. No second payout or refund.    |
| Later REFUNDED / reversal    | Post-payment exception        | Separate review and bookkeeping correction.         |

PayPal: item status definitions · Official Payouts example includes failed/unclaimed items inside a SUCCESS batch.

# Refund only after clear no-payment evidence.

A treasury refund is a new signed transfer. The buyer's spending mandate does not authorize this reverse movement.

## Refund policy

- Original buyer only.
- Same mint and token program.
- Exact original token base units.
- One refund operation per original receipt.
- Terminal failure or verified returned funds.

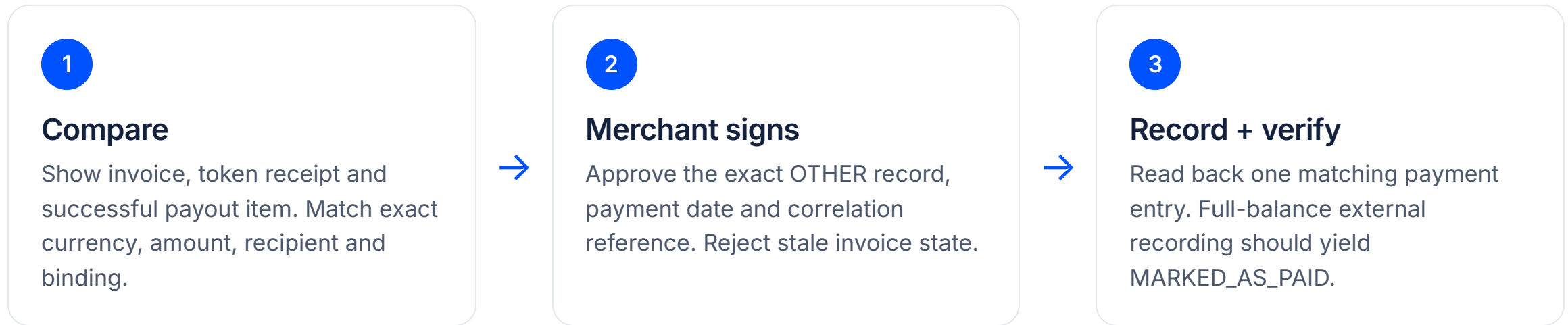
## Execution policy

- Exclude any payout worker before refund.
- Persist signed transaction + signature first.
- Broadcast, then reconcile chain history.
- Timeout becomes **refund unknown**.
- No replacement until nonlanding is proven and the original cannot land.

The original receipt stays. Refund does not restore mandate amount\_spent or count; further spending needs valid remaining authority or new owner approval.

# Record the payout after success. Read the entry back.

A bookkeeping error cannot justify sending more money or refunding a completed payout.



Keep **token settled**, **payout succeeded**, **invoice recorded** and **delivery evidence** as separate facts. An unknown record response triggers read-only recovery.

# The invoice owner and payout sender are different roles.

A merchant display name or email field is not proof of authority over a PayPal account.

| ROLE                             | REQUIRED EVIDENCE BEFORE IMPLEMENTATION                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Merchant invoice owner</b>    | Authorized sandbox credentials or supported merchant authorization; prove create/send/read/record on that merchant's invoice. |
| <b>ChainPay payout sender</b>    | Separate prefunded sandbox business account and server-side Payouts credentials.                                              |
| <b>Merchant payout recipient</b> | Enrolled account identifier bound to the merchant's signed terms; verify expected recipient and item details in the sandbox.  |
| <b>Buyer + treasury</b>          | Verified mandate owner and treasury destination; signer access isolated from agent tools.                                     |

Do not assume a single app can write any merchant invoice. Prove the chosen two-account setup before building the full flow.

# Verify the event. Read the current state.

Treat a notification as evidence to verify and a reason to fetch current provider state.

## Authenticate

Verify PayPal signatures against the configured app webhook ID. Scope every event to sandbox and the intended account.

## Accept durably

Store verified events before acknowledging them; deduplicate event IDs and tolerate retries and reordered arrival.

## Reconcile item

Read current payout item and invoice. Match IDs and canonical intent. Resume only the action allowed by the durable state machine.

A periodic recovery job and manual "Check status" action cover missed events. Neither action is a new payout or refund command.

# Prove the provider contract before connecting the treasury.

Suggested owners: Kwasi — payment policy and backend. Dre — approval UI, receipt evidence and demo.

| PHASE                | CONCRETE DELIVERABLE                                                                         | EXIT EVIDENCE                        |
|----------------------|----------------------------------------------------------------------------------------------|--------------------------------------|
| 1 • Scope + accounts | Agree custody terms, deployment identity, invoice owner, payout sender and recipient.        | Account map and signed-off contract. |
| 2 • Sandbox spike    | Invoice lifecycle; payout item success/failure; OTHER record/readback; webhook verification. | Sanitized real sandbox responses.    |
| 3 • Durable policy   | Immutable binding, claims, dispatch recovery, exact refund state machine.                    | Replay, concurrency and crash tests. |
| 4 • Devnet loop      | Existing mandate → treasury receipt → gated payout → merchant record.                        | Linked chain + provider evidence.    |
| 5 • Failure demo     | Terminal payout failure, exact refund once, unresolved outcome waits.                        | Three reproducible acceptance runs.  |

Re-estimate after Phase 2. The old 19–31 engineer-day estimate did not include treasury payouts or compensation.

Proposal sequence, not a delivery-date commitment. Finance assistants and sponsor extras follow the verified core loop.

# Show success, a safe refund and an honest wait.

No “works on Devnet” claim until the evidence package survives repeated and concurrent requests.

## Run A · Successful settlement

One signed binding → finalized token receipt → matching payout item SUCCESS → one exact invoice payment entry.

## Run B · Terminal payout failure

Tokens reach treasury; verified failure leads to one same-buyer exact-amount refund. Replay events and confirm no additional transfer.

## Run C · Outcome unknown

Lose the provider or chain response; show “checking outcome.” Resolve the original operation without a second payout or refund.

Use only sandbox accounts and Devnet tokens. Label any simulated provider failure clearly; do not present it as actual PayPal settlement.

# Five choices make the proposal implementable.

The research supports a sandbox spike. These choices determine its exact contract.

| DECISION                                              | RECOMMENDED STARTING POINT                                                                                   |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> <b>Treasury model</b>        | Accept temporary test-token custody and explicit compensation responsibilities for sandbox only.             |
| <input type="checkbox"/> <b>Deployment continuity</b> | Keep approved Axum + Convex. Verify target deployment, durable claims and existing-data continuity.          |
| <input type="checkbox"/> <b>Account mapping</b>       | Separate controlled invoice merchant and ChainPay payout sender; bind the recipient before debit.            |
| <input type="checkbox"/> <b>Exact demo terms</b>      | One USD invoice, one Devnet token, full payment only; explicit token amount, fiat amount and fee allocation. |
| <input type="checkbox"/> <b>Invoice race policy</b>   | Controlled demo accounts plus outside-payment detection. No claim of atomic cross-system settlement.         |

Planning approval does not authorize financial actions. Production eligibility, real funds and a conversion bridge are separate work.

# This replaces the old payment story, not the original files.

The nine-step structure survives. Its money movement and recovery guarantees are now more precise.

## OCTOBER 3 FRAMING

**Money moves only on Solana**

**Receipt proves merchant payment**

**Bookkeeping recovery only**

**Idempotency prevents all duplicates**

**Fixed estimate and deployment assumptions**

## OCTOBER 5 REVISION

Two movements: token treasury transfer + prefunded PayPal sandbox payout.

Receipt proves treasury payment; PayPal item proves recipient payout.

Payout recovery, treasury compensation and bookkeeping recovery are distinct.

Documented provider retention + permanent local claims + unknown-outcome discipline.

Re-estimate after the spike; verify deployment and existing-data continuity.

Next concrete deliverable: an account-mapped sandbox contract spike, followed by the smallest complete Devnet success + failure demonstration.

Revision based on the partner conversation and official PayPal documentation retrieved 5 October 2026.

# Provider facts are sourced; system guarantees still need proof.

Official PayPal pages retrieved 5 October 2026. Linked research digest contains the complete recovery contract.

## PAYOUTS

[PayPal: sandbox setup and live enablement](#)

---

[PayPal: Payouts integration](#)

---

[PayPal: duplicate request window](#)

---

[PayPal: item status definitions](#)

---

[PayPal: balances API freshness](#)

---

## INVOICES + EVENTS

[PayPal: Invoicing reference](#)

---

[PayPal: invoice lifecycle](#)

---

[PayPal: webhook verification](#)

---

[Payout webhook event names](#)

---

[Invoice payments schema: OTHER + explicit amount](#)

---

Source notes: partner proposal + October 3 deck. Stale UI mockups and no-payout UML diagrams are omitted because they describe the prior flow. This revision does not claim implementation or production approval.